

1

What are React and React Native?

Welcome to the magical world of **React** and **React Native**. I hope to make you feel at home here. It's okay if this is your very first contact with these frameworks, or you may have played around with them a little bit already. This book will focus on **managing state** in **React Native apps**, but we will start by going over the basics.

If React and React Native were people, the first one would be the parent of the second one. You can focus on the child framework, but you will find great benefits in getting to know the “parent” of React Native – **ReactJS**.

We will start this journey by going over the history of ReactJS and specifically, why it was created. We will then continue our study of ReactJS by looking into what it means to *think in React*

or to have the *React mindset*. Once we're familiar with ReactJS, we will try to understand what **cross-platform software development** means and what place React Native holds in the cross-platform development ecosystem. For understanding the ecosystem, we will concentrate on React Native itself, its brief history, and its current state. We will finish our tour with a handful of examples of native apps written in React Native.

In this chapter, we will cover the following topics:

- Understanding the history of ReactJS
- Thinking in React (the React mindset)
- Understanding cross-platform software development
- Going over the history of React Native
- Reviewing examples of popular apps using React Native

By the end of this chapter, you will have high-level knowledge of React and React Native. You will also understand their place in the software development ecosystem.

Understanding the history of ReactJS

In this section, we will briefly look into the history of ReactJS. If you're not interested in this particular topic, feel free to skip

this section and go straight to *Thinking in React*. Knowing the history of a framework is not compulsory for using it. If you prefer condensed knowledge served in a YouTube pill, I highly recommend watching a 10-minute video called *The Story of React*, published on YouTube by *uidotdev*.

The predecessors

Did you know that the first website ever created is still live?

You can find it here:

<http://info.cern.ch/hypertext/WWW/TheProject.html>. It was created in 1991! A lot has changed since then. For starters, web developers wanted to change their websites' appearance, hence CSS was created. A few years later those same web developers wanted to see more interactivity on their now beautiful websites. This is when **JavaScript** found its place on the internet. But as we know, the web never stops evolving. Heavy usage of JavaScript led to the creation of libraries such as jQuery, BackboneJS, and Ember. Each library's creators learned lessons from their competitors. They made decisions that led to creating very different developer experiences. The developers had their preferences and little wars over which library is better.

There is no right answer to this question. What is certain, however, is that user experience on websites evolved, no matter

which library was used behind the scenes. Websites became much more interactive and adaptable to the user's screen size. For example, it is common practice today to create separate menus for mobile views and desktop views. This can be achieved with JavaScript, or CSS alone. This user experience shift could not take place without the evolution of JavaScript **open source** libraries.

After a few years of adding more and more separate bits of JavaScript to websites, it was time for a more holistic solution. The first breakthrough came from Google, with **AngularJS**. AngularJS, officially released in 2010, was different from other solutions on the market at that time. This was not just another library; this was a framework. Developers were able to create complex interactions quickly, and they were no longer afraid that any change to their JavaScript files could break the entire page. I don't want to go into the implementational details of AngularJS. After all, that is not the focus of this book. In broad strokes, AngularJS introduced special HTML attributes that were observed by the framework running in the background. As you may imagine, when JavaScript is observing dozens or even hundreds of elements and events, it will slow down. So, the user experience is suffering, and the world is ready for another JavaScript revolution. Google thought they would remain king of the hill with their Angular version 2, but in 2013, Facebook developers announced the release of ReactJS.

And then there was React

ReactJS was presented specifically as a **user interface (UI)** library. It was conceived to be used for end user interactions on websites. It also used **JSX** – an extension to JavaScript created for React. Many developers reacted, pun intended, angrily to this new syntax. I would say though, that angry reactions are not unexpected in the world of tech. Any new technological solution has to weather the storm of angry Reddit posts saying that it's ugly, useless, or simply terrible. Luckily for us, ReactJS developers did not stop working on their open source framework because of this initial negative reaction. Furthermore, developers who got to know ReactJS became its advocates. Why, you may ask, did ReactJS stand the test of time, while Angular hasn't? I believe it has to do with the high-level mindset of the framework. ReactJS proposes elegant, simple solutions while staying completely configurable to any needs. I will go further into this mindset idea in the next section.

Going back to our history lesson! We're in 2013, and ReactJS has entered the scene with a bang. Many people hate it, but others use it for more and more complex websites. And it turns out, unfortunately, that ReactJS does not scale well. Your React components use state and props. If the parent creates a state, which needs to be read four to five components lower in the hierarchy, you encounter something that is dubbed **prop**

drilling. Prop drilling means that the developer has to pass the necessary prop through many parent components in order to get to the final child that needs to read it. This process is irritating and boring, at the same time! This is when the first state management library was created – **Redux**. We will talk in detail about Redux and other state management libraries in the next chapters.

As of writing this book, ReactJS is one of the most popular JavaScript libraries. It evolves constantly, and its maintainers are open to public discussions and suggestions. In 2019, they introduced hooks and context. These two React utilities can cover a lot of your state management needs. They were created because the React team realized that developers using React needed an improvement in the state management area.

A few years before the introduction of hooks and context, specifically in 2015, Facebook developers released React Native. The true hero of this book! But let's not get ahead of ourselves. At this moment, it is important that you understand the basic concepts of React. Let's move on to the React mindset.

Thinking in React (the React mindset)

The official ReactJS docs include a chapter called *Thinking in React*: <https://reactjs.org/docs/getting-started.html#thinking-in-react>.

IMPORTANT NOTE

*Many React users credit reading *Thinking in React* (<https://reactjs.org/docs/thinking-in-react.html>) as the moment React finally clicked for them. It's probably the oldest React walk-through but it's still just as relevant.*

Let's try and capture the most important, and still relevant, parts of that article.

First of all, when we create a website with ReactJS, we need to think about how we will construct our components. Not HTML blocks, not DOM elements, but components. Ideally, each component will be a separate entity, which either creates a state or consumes props, or sometimes both. The component is the smallest part of our app, just like atoms are the smallest parts of our world.

Okay, I realize atoms can be further divided into neutrons, protons, and electrons. And **ReactJS components** can be divided into parts that handle the logic and the actual rendering. However, both atoms and ReactJS components are the basic building blocks in their respective realms.

Now that we have our components imagined, we need to know how they should interact with each other. Let's go back to the ReactJS docs, where we will find a great chapter, *Composition vs. Inheritance*: <https://reactjs.org/docs/composition-vs-inheritance.html>.

This article is very clear in stating that ReactJS components should be composed, and not stacked in a strict hierarchy. This basically means that any child component should be created in a way that it could be reused by other parent components throughout the app. This promotes the high reusability of atomic components, and at the same time, reduces the amount of code needed to create an application.

Now that we have the theory down, let's move on to specifics. How do we compose ReactJS components in practice? By using state and props. What are those, you may ask? Well, I'll be glad to explain!

Both **state** and **props** (short for properties) are plain JavaScript objects. The big difference between them is that props are read-only, while state can be changed within the component that manages it. State is the source of truth, while props are the representations of the current state of the application. Let's take a look at a minimal code example:


```
import React, { useState } from "react";
const PrettyButton = ({ updateCount, count }) => {
  return (
    <button onClick={updateCount}>This was clicked {count} of
    times</button>
  );
};
export default function App() {
  const [counter, updateCount] = useState(0);
  const handleClick = () => {
    updateCount(counter + 1);
  };
  return (
    <div>
      <h1>Hello There!</h1>
      <PrettyButton count={counter} updateCount={handleClick} />
    </div>
  );
}
```

You can play with this sample code online thanks to this CodeSandbox: <https://codesandbox.io/s/admiring-fire-68k94x?file=/src/App.js>.

From the preceding code example, you can see that the **App** component creates the counter state, and the function responsible for updating it. **PrettyButton** consumes this state in the form of props. **PrettyButton** cannot change the value of **counter** or **updateCounter** directly.

If we were to write another parent component that needed to use **PrettyButton**, it would need to create its own **counter** and **updateCounter** states. And thanks to that, every instance of **PrettyButton** we may want to use in our web app will be independent of the others.

We may also find ourselves importing multiple child components in the main **App** component. This is totally natural. We may have an app with a button, a text, and a modal, all of which need to display the number of times the button was clicked. All we need to do is add the necessary components to the parent and pass the **counter** prop. The state is mutated *only* in the parent and then fed to the children.

Now we arrive at the moment where we need to decide which component should handle the state change. In our simple code example, the answer is obvious: we have only one parent. In the real world, this question may be much more difficult to answer. Luckily for us, we will look at state management strategies throughout this entire book. I hope, after reading this book, that you will be well equipped to choose the best place to store and manage your application state in your React Native app.

In the previous section, we went over high-level aspects of writing code in ReactJS. It's good to keep in mind the patterns

we looked at, as they are just as useful in React Native development. And since we're familiar with ReactJS, we are ready to dive into the world of native apps written in JavaScript.

Understanding cross-platform software development

Before talking about React Native, we need to go over the landscape of mobile app development.

It is quite obvious that mobile apps can be created using native platform programming languages. The ones considered most modern are Swift, for iOS development, and Kotlin, for Android development. Many developers still use Objective-C and Java, respectively. However, when the market of mobile phones settled down with the two giants, Apple and Google, it was tempting to create solutions that could be written once for both platforms. Similarly, for websites, which can be opened in any browser, why can't we have apps that can be run on any device?

Looking for this mythical cross-platform solution was enticing to many companies. They were hiring separate teams from iOS and Android to end up with apps that do not look and feel the same.

The software development world is vast, and we can find many solutions to a single problem. Cross-platform development is not an exception to this rule. If you google **cross-platform apps**, you will find a solution from Microsoft, called **Xamarin**. You will also find **Flutter**, written in a language called **Dart**. And finally, you will find many solutions based on JavaScript. One of the first meaningful players was **Ionic**. Ionic is a framework, built in 2013, for development in AngularJS, and it uses **Apache Cordova** behind the scenes. Ionic developers build their apps using the exact same syntax they would use to create a website. At build time, a native app **wrapper** with a single WebView is created. The Ionic code is run inside this WebView. Given this structure, many people call Ionic apps **hybrid apps** to differentiate them from **cross-platform apps**.

React Native is a completely different solution. In its case, code is compiled into a complete native app. JavaScript code runs in the app and communicates with the phone's native modules through a **bridge**. But where did React Native come from, you may ask?

Let's dive into that topic in our next section.

Going over the history of React Native

Back in 2012, Facebook announced they were becoming a *mobile-first company*. Facebook realized its users spend more time on their phones than on computers. They needed to have their websites and apps working seamlessly on smart devices. However, the majority of Facebook engineers were web developers. The company started researching options to reuse the knowledge of those web developers for mobile development. After trying out a few different ideas, they didn't want to follow in the footsteps of Ionic, enclosing the apps inside WebViews. They needed something new.

That is when a developer named *Christopher Chedeau* made his mark on the history of software development. He teamed up with *Jordan Walke*, *Ashwin Bharambe*, and *Lin He* for an internal Facebook hackathon. Basing their work on the first attempts done by Jordan – who, by this time, had been able to generate *UILabel* in iOS from JavaScript – they created a working prototype that could generate native UI elements from JavaScript on the user device. And it took them only 2 days!

*THE HISTORY OF REACT NATIVE: FACEBOOK'S OPEN
SOURCE APP DEVELOPMENT FRAMEWORK*

You can read the article here:

<https://www.techaheadcorp.com/blog/history-of-react-native/>.

After this initial success, Jordan and Christopher were able to continue working on their new product, named React Native, with an entire team of engineers.

After 3 years, they were ready to present what they had to the world. The official announcement for React Native took place at ReactJS Conf in 2015. This was the first ReactJS Conf, and React Native was presented during the keynote! That's how much faith Facebook had in this framework. I encourage you to check out the talk; you can find a link in the official ReactJS docs at <https://reactjs.org/blog/2015/02/18/react-conf-roundup-2015.html>.

Since 2015, React Native has grown and changed a lot. Some changes, such as the introduction of hooks and context, were simple follow-ups to changes happening in ReactJS. In other cases, changes were motivated by the community or proposed by the maintainers of the framework. React Native on github.com has a whole section called *Discussions and Proposals* (<https://github.com/react-native-community/discussions-and-proposals>). Everyone is welcome to add anything they would like to discuss on the topic of React

Native implementations, ecosystems, and so on. This board is a great resource for what is currently going on and what may be expected to happen in the future. One of the first issues on this board, the sixth issue to be exact, was a proposition for a *Lean Core*. By this time, React Native has been in the wild for at least 3 years and it has grown a lot. The framework has included implementations of UI details such as Switch, or native functionalities such as push notifications. One of the core maintainers of the repo proposed that all code that is not absolutely necessary be removed from the main package. You can read more details on *Lean Core* here: <https://github.com/react-native-community/discussions-and-proposals/issues/6>.

Of course, answering the question of “what is necessary” and “what isn’t” is not easy. The *Lean Core* took a few months of discussions and breaking changes. The shape of the main React Native package today represents the results of this effort.

In the meantime, the *Lean Core initiative* energized the community to go ahead and create their own libraries, which could be useful for React Native apps. As of writing this book, there are hundreds of libraries to choose from when you decide to create a React Native app. There are UI libraries, navigation libraries, async storage management libraries, and many more. This is a blessing and a curse because not every library is well-written and maintained correctly. You can, unfortunately, hap-

pen to use something that may break your app in the future. So before running to your terminal and typing **yarn add**, you may want to use the React Native directory:

<https://reactnative.directory>. This website provides metrics on open source libraries, which are very helpful when you want to add a good dependency to your project.

There are a few libraries that stand out so much, and they are considered to be *recommended* for React Native projects. Those libraries are usually pretty mature and well-maintained. One example is **React Navigation**, the go-to library for apps that need anything more than one screen. **React Native Testing Library** is a library officially coupled with Kent C. Dodd's **React Testing Library**. **Reanimated** is an animation library, which achieves better performance than any of its competitors.

An important part of the React Native ecosystem is **Expo**: <https://expo.dev/>. Expo is both a framework and a platform for React Native applications. It offers its users a set of tools useful for developing, building, and deploying apps.

What does that mean specifically? Expo is a thin layer on top of React Native, aimed at making the life of developers easier. If writing an app in React Native was like eating a grilled steak with your hands, Expo would be like eating Filet Mignon with

a baked potato and a side of Caesar salad. In a fancy restaurant. You may very well prefer the former, but you cannot deny the obvious advantages of the latter. If you decide to use Expo, you will find local environment setup instructions in the official React Native docs:

<https://reactnative.dev/docs/environment-setup>. Once the app is set up, you will be able to take advantage of the many components created and maintained by the Expo team. This way, you may save yourself a few headaches and performance problems. When you're ready to show your app to the world, you can upload your app bundle to the Expo website and use it for testing and deployment. As you can see, Expo is a very versatile tool.

Now that we're up to speed with the history and the current state of React Native, let's move on to looking at some real-world apps that use it.

Reviewing examples of popular apps using React Native

Now that we know a little bit about React Native, it's time to get excited about it. A great way to get excited about a new tech-

nology is to look at what that technology has already been used for. This is also a good strategy when you must decide to use a particular technology.

The obvious example comes from **Meta** – the birthplace of React Native. The very first implementations of ReactJS took place in Facebook Ads. It is fitting that React Native is used for that same feature on mobile devices. Facebook’s mobile app is not entirely created with React Native, but some parts of it use it. That means the Facebook app is a React Native *brownfield* app. The opposite of that is apps written in React Native alone, and that sort of app is called *greenfield*.

While we’re in the Metaverse, I will mention that the Instagram app uses React Native, as does the Oculus app.

Don’t worry, Meta is not the only notable company using React Native. Discord not only uses React Native for their app, but they also write blog posts about how they maintain their app. In this Medium article, <https://blog.discord.com/how-discord-achieves-native-ios-performance-with-react-native-390c84dcd502>, the Discord team states that they adopted React Native as soon as it was open sourced, and they are still happy with their decision years later.

Shopify is another big player in the React Native ecosystem. They have an article on their blog entitled *React Native is the*

Future of Mobile at Shopify: <https://shopify.engineering/react-native-future-mobile-shopify>. Shopify engineers also write more technical articles, for example, about accessibility: <https://www.shopify.com/partners/blog/react-native-accessibility>.

The website-builder giant **Wix** is also active in the React Native world. They have also written about their adventure with React Native (<https://medium.com/wix-engineering/react-native-at-wix-the-architecture-db6361764da6>) but they also create open source libraries, for example, this UI kit: <https://github.com/wix/react-native-ui-lib>.

Circling back to listing specific apps built with React Native, I have to mention **Coinbase**. Managing users' finances in a reliable manner is the top priority for this crypto market leader. They analyzed, iterated, and landed on using React Native as their main mobile technology. You can read their article about the transition from native technologies on their blog: <https://blog.coinbase.com/announcing-coinbases-successful-transition-to-react-native-af4c591df971>.

You may have heard of companies such as Tesla, Walmart, Salesforce, Bloomberg, and Vogue. You may have used apps such as Uber Eats, Artsy, Words with Friends, and SoundCloud Pulse. What do they have in common? Surprise! (Not really.)

They all use React Native. You can find even more examples with links to articles in the React Native showcase:

<https://reactnative.dev/showcase>.

Not all React Native stories are success stories, though. One famous case (by *famous*, I mean it was tweeted about for a few days) is Airbnb. Airbnb's website uses ReactJS, so it was logical for them to try React Native for their mobile app. After a few years of development, they hit development roadblocks and performance issues. Their app consists of a very big map that needs to work perfectly. The developers working on the app often needed help from React Native developers, which was a bottleneck for this web-technology-focused company. They announced their divorce from React Native in 2018:

<https://medium.com/airbnb-engineering/sunsetting-react-native-1868ba28e30a>. Luckily, they still develop their amazing animation library, **Lottie** (<http://airbnb.io/lottie/#/>), which can be used in React Native apps.

Summary

Oof! That was a lot of theory for a programming book, right? However, even if you found it a little dry, I strongly believe this theoretical knowledge will be very useful for the next chapter. We have learned a little bit about the history of web development and about the motivations of the creators of both ReactJS

and React Native. Knowing all of this will let us understand the ideas behind different state management solutions. In the next chapter, we will jump into the most basic way of managing state in a React Native app: with hooks and context.

